

AI-Powered Detailed Project Report (DPR) Quality Assessment and Risk Analysis System for MDoNER

Rama Ranjini
Assistant Professor, Dept. of CSE
Sri Krishna College of Engineering and
Technology
Coimbatore, India

Aryan Sharma
Dept. of CSE
Sri Krishna College of Engineering and
Technology
Coimbatore, India

Ahil Aadithya R S
Dept. of CSE
Sri Krishna College of Engineering and
Technology
Coimbatore, India

Arpit Raina
Dept. of CSE
Sri Krishna College of Engineering and
Technology
Coimbatore, India

Dhayanandh A
Dept. of CSE
Sri Krishna College of Engineering and
Technology
Coimbatore, India

Abstract—Detailed Project Reports (DPRs) consolidate the technical, financial, schedule, risk, environmental, and administrative information required to evaluate proposed projects, and manual review of such reports is time-consuming and inconsistent across reviewers. This paper presents an AI-assisted DPR quality assessment and risk-analysis system that automates preliminary document screening while preserving transparent, deterministic risk scoring. The implemented pipeline extracts page-level text from DPR PDFs using PyMuPDF, detects sections through Table-of-Contents analysis, classifies sections semantically with Gemini 2.5 Flash, evaluates completeness through evidence-based checklists, and performs LLM-assisted quality assessment. The resulting structured features feed a rule-based risk engine that computes delay, cost, and approval risk, after which an LLM generates a human-readable explanation. The system is delivered through a FastAPI backend, a PostgreSQL database, and a React frontend supporting authentication, DPR upload, assessment tracking, dashboard analytics, and report management. A demonstration run on a real DPR produced an overall completeness score of 57.29 across eight detected sections. The system is best positioned as a decision-support prototype rather than a statistically validated predictor; historical outcome-based prediction, OCR, numerical validation, and authoritative MDoNER compliance checking remain future work.

Keywords—Detailed Project Report, document intelligence, large language model, completeness assessment, quality assessment, risk analysis, Gemini, PyMuPDF, explainable AI, MDoNER.

I. INTRODUCTION

Detailed Project Reports are central documents in project planning and appraisal because they consolidate the technical description, objectives, financial requirements, implementation schedule, risks, environmental considerations, and supporting justification of a proposed project. In practical review environments, DPRs can be lengthy and may use different section names and document structures, so reviewers spend substantial effort locating relevant information, checking whether expected sections are present, and identifying weak or incomplete content.

Conventional review also makes it difficult to obtain a standardized preliminary assessment across a large number of DPRs, since a reviewer must inspect the Table of Contents, navigate page ranges, compare technical and financial sections, judge the quality of explanations, and identify possible implementation risks. These activities are valuable but repetitive, and their consistency depends on the individual reviewer and the time available.

Recent advances in document processing and large language models provide an opportunity to automate the preliminary stages of DPR assessment. A fully LLM-driven approach, however, can make numerical decisions difficult to reproduce or audit. The proposed system therefore follows a hybrid architecture in which LLMs perform semantic interpretation and qualitative assessment while deterministic components perform evidence-based completeness scoring, feature construction, and numerical risk calculation.

The implemented system accepts a DPR in PDF format and processes it sequentially: PyMuPDF extracts page-level text, a section detector identifies the Table of Contents and maps logical sections to physical pages, and Gemini 2.5 Flash classifies sections into a standardized taxonomy. Completeness is assessed using predefined checklist items and evidence snippets, while quality is evaluated across clarity, technical depth, consistency, organization, professionalism, and evidence and justification. The resulting structured features are passed to an explainable risk engine.

The main contributions are fourfold: (1) an end-to-end document-understanding pipeline for DPR PDFs; (2) semantic section classification combined with deterministic evidence-based completeness assessment; (3) integration of LLM-based qualitative quality assessment with a transparent, rule-based risk engine covering delay, cost, and approval risks; and (4) a complete FastAPI, PostgreSQL, and React application exposing this analytical workflow to authenticated users.

II. RELATED WORK AND RESEARCH GAP

A. Conventional DPR Review

DPR appraisal traditionally relies on manual inspection of document structure and content. Reviewers examine whether required topics are covered and whether the available information is sufficiently detailed for technical and administrative evaluation. This is effective with experienced reviewers but becomes resource intensive for large document collections and difficult to standardize. Recurring review needs include automated ingestion, structural analysis, semantic section classification, completeness assessment, qualitative quality assessment, feature generation, risk scoring, and explainable reporting, which together motivate an integrated platform rather than isolated utilities.

B. Existing AI-Assisted Approaches

Language-model-based systems suit semantic interpretation because DPR headings are not guaranteed to follow a fixed convention; headings such as Capital Cost, Financial Estimate, Project Cost, and Estimated Cost can all refer to related financial content, and a semantic classifier can map such variations into a standardized category while preserving the original content. At the same time, numerical decisions should not be delegated entirely to an LLM when reproducibility and auditability matter. Prior work on delay and cost-risk prediction has explored ensemble and gradient-boosted models such as XGBoost, CatBoost, and LightGBM [1], [2], [3], and explainability frameworks for such models are increasingly emphasized in the literature [4]. The implemented system separates semantic interpretation from deterministic scoring: the LLM identifies and evaluates content, while rule-based modules convert structured assessment features into repeatable risk indicators.

C. Research Gap

A gap remains between the broad objective of machine-learning-based risk prediction and what is practical with the available project data. The implementation does not contain a historical dataset linking DPR characteristics with actual project delay, cost overrun, or approval outcomes, so the present risk engine is a knowledge- and rule-based scoring mechanism rather than a trained predictive model. Additional gaps include the absence of OCR for scanned DPRs, authoritative MDoNER guideline retrieval and clause-level compliance checking, numerical financial validation, deterministic cross-section consistency analysis, and a statistically validated evaluation dataset. These limitations define the boundary of the current system and motivate the roadmap discussed in Section VIII.

III. PROPOSED DPR ANALYSIS FRAMEWORK

A. System Overview

The proposed framework is organized as a modular pipeline connecting document ingestion, structural analysis, semantic classification, completeness assessment, quality assessment, feature construction, risk analysis, explanation generation, and application-layer reporting. The risk engine is kept independent of raw PDF text and instead consumes structured signals produced by earlier stages, which makes the numerical risk calculation deterministic once its inputs are fixed. Authenticated users can upload DPR PDFs, start an assessment, monitor processing progress, view dashboard statistics, retrieve completed reports, inspect risk drivers and recommendations, and delete reports.

B. System Architecture

The AI subsystem is implemented primarily in Python. PyMuPDF handles PDF processing, Google Gemini 2.5 Flash provides language-model capabilities, and Google ADK components organize functional agents. The application backend is implemented with FastAPI and SQLAlchemy, while PostgreSQL stores users, sessions, DPR metadata, and structured assessment results. The frontend is a React single-page application built with React Router, Axios, Tailwind CSS, Lucide React, and React Hot Toast.

Module	Primary Function	Type
PDF Reader	Extract page-level and complete document text	Deterministic
Section Detector	Identify TOC and map page ranges	Hybrid
Section Classifier	Map arbitrary headings to standard categories	LLM-assisted
Completeness Checker	Detect checklist items and evidence	Deterministic
Quality Assessor	Score qualitative section dimensions	LLM-assisted
Feature Builder	Construct structured risk features	Deterministic
Risk Analyzer	Calculate delay, cost, approval and overall risk	Rule-based
Risk Explainer	Generate natural-language justification	LLM-assisted

TABLE I. MAJOR COMPONENTS OF THE DPR ANALYSIS FRAMEWORK

C. Processing Workflow

The analytical sequence begins with a DPR PDF supplied by an authenticated user. The PDF reader extracts text from each page and preserves page boundaries; the section detector searches the first five pages for a Table of Contents and asks Gemini to convert it into structured section titles and page ranges, which are mapped back to the page-level text. Each detected section is passed to the semantic classifier, which assigns one category from a standardized taxonomy—Introduction, Objectives, Technical, Financial, Budget, Timeline, Risk, Environmental, Legal, Appendix, or Other—together with a confidence estimate. The classified sections then branch into completeness and quality assessment, whose outputs are combined by the Feature Builder before risk analysis.

The FastAPI backend exposes dedicated authentication, DPR, report, and dashboard routes. User authentication uses official IDs, bcrypt password verification, server-side sessions, HTTP-only cookies, session expiry, and suspension checks. DPR upload validates content type and extension, sanitizes filenames, prevents path traversal, limits file size to 50 MB, and writes files in 1 MB chunks. PostgreSQL is accessed through SQLAlchemy across four principal entities—user auth, user details, sessions, and dpr files—with the complete AI assessment persisted in a JSONB field so structured results can be stored without a separate table for every attribute.

IV. AI-ASSISTED DPR QUALITY ASSESSMENT

A. PDF Document Processing

The PDF reader processes every page individually, storing a mapping from page number to extracted text while also constructing a complete document representation; both are needed because downstream section extraction depends on

physical page boundaries. PyMuPDF provides a mature, computationally inexpensive extraction layer for digitally generated PDFs containing selectable text, but the current layer does not include OCR, table reconstruction, layout-aware extraction, image interpretation, or handwriting recognition—a limitation for scanned or image-heavy government DPRs.

B. Section Detection and Semantic Classification

The section detector locates the Table of Contents within the first five pages using content-related keywords; once identified, Gemini converts it into structured entries with a title, start page, and end page, the latter inferred from the start page of the following section. A page-offset mechanism accounts for differences between logical DPR page numbers and physical PDF page numbers when a recognizable chapter/introduction pattern exists. Semantic classification then assigns each section exactly one standardized category with a confidence value; this flexibility is useful because DPRs use inconsistent terminology, although classifier confidence remains an LLM-generated estimate that has not been calibrated against a labeled dataset.

C. Deterministic Completeness Assessment

Completeness is assessed through category-specific checklists—for example, Introduction covers project background, objectives, need of project, location, scope, and stakeholders, while Risk covers risk identification and a mitigation plan. The checker searches section content for predefined keywords, extracts surrounding evidence, and marks each checklist item as present or missing; a section score is obtained from the proportion of present checklist weights, as in (1). This evidence-based design is more reproducible than asking an LLM to assign completeness directly, since each decision can be traced to a detected evidence snippet.

$$\text{Section Completeness} = (\sum \text{Present Item Weights} / \sum \text{All Item Weights}) \times 100$$

D. Quality Assessment

The quality module depends substantially on Gemini. Each classified section is assessed across six dimensions—clarity, technical depth, consistency, organization, professionalism, and evidence and justification—with a score from 0 to 100 for each, together with strengths, weaknesses, issues, and reasoning. The result is useful for producing structured, expert-like feedback, but should be interpreted as an LLM-based qualitative assessment rather than a statistically calibrated metric, since no deterministic formula currently combines the six dimensions into an overall score and no human ground-truth benchmark is available.

E. Feature Engineering and Risk Analysis

The Feature Builder converts completeness and quality reports into a compact representation for the risk engine, including overall completeness, overall quality, missing sections, section counts, and category-level quality features for technical, financial, budget, timeline, risk, environmental, and legal content. The risk engine operates on these structured features rather than raw PDF text, computing delay, cost, and approval risk independently; each category begins from a baseline and applies defined adjustments when relevant conditions are detected (e.g., missing timeline, low technical quality), with the result clipped to the 0–100 range. The

overall score is given by (2), which is converted into three risk classes: 0–39.99 Low, 40–69.99 Medium, and 70–100 High.

$$R = 0.40 R_{\text{delay}} + 0.35 R_{\text{cost}} + 0.25 R_{\text{approval}}$$

(2). This provides a transparent, reproducible decision rule because the contribution of each component can be inspected, and it should be described as knowledge- or rule-based risk scoring rather than machine-learning risk prediction, since it does not learn from historical project outcomes.

F. Risk Explainability and Failure Handling

The numerical risk score is produced deterministically, after which Gemini receives the computed scores, risk drivers, recommendations, feature summary, and rule breakdown to generate a natural-language explanation; the LLM is explicitly instructed not to modify numerical scores or introduce new facts, which keeps the language model responsible for communication rather than for the numerical decision itself. Retry handling covers common transient service errors, JSON extraction supports plain and markdown-embedded JSON, and deterministic fallback reports are produced when parsing fails, reducing the likelihood that malformed LLM output terminates the pipeline.

V. APPLICATION AND USER INTERFACE

A. React Application and Backend Services

The user-facing application is a React single-page application with public and protected routes separated through an authenticated shell. Main pages include a landing page, official login page, dashboard, DPR assessment page, reports page, detailed report page, and server-health page, with reusable components (StatCard, SkeletonCard, QuickActionCard, ActivityItem, SectionHeader, EmptyState) providing consistent presentation. A centralized Axios instance, configured with a 10-second timeout and credentialed requests, connects every page to the FastAPI backend.

B. Assessment Workflow

The assessment page supports PDF selection, drag-and-drop upload, file replacement, and client-side validation of PDF format and the 50 MB size limit. After upload, the frontend calls the upload endpoint, stores the returned file identifier, and invokes the analysis endpoint. During processing, the interface presents a pipeline—Upload Complete, Extracting Text, Identifying Sections, Compliance Verification, Risk Prediction, Report Generation, and Assessment Complete—showing the current stage, progress percentage, estimated remaining time, and timestamped activity logs.

C. Dashboard and Report Management

The dashboard provides an operational overview: number of assessed DPRs, pending assessments, high-risk projects, average risk score, average completeness, average quality, and average AI confidence, alongside risk distribution, recent evaluated DPRs, recent activity, and automatically generated operational insights. The reports interface offers table and card views with search by filename, filtering by risk level and date, and sorting by recency; each report can be opened for a detailed view or deleted, with confirmation and toast-based feedback throughout.

Frontend Operation	Backend Endpoint	Purpose
Login	POST /auth/login	Authenticate official user
Session check	GET /auth/me	Verify current session
DPR upload	POST /dpr/upload	Store DPR PDF
Start assessment	POST /dpr/{file_id}/analyze	Invoke AI assessment
Dashboard	GET /dashboard	Retrieve analytics
Reports	GET /dpr/reports	List completed assessments
Report detail	GET /dpr/reports/{file_id}	Retrieve one assessment
Delete report	DELETE /dpr/reports/{file_id}	Delete report and file

TABLE II. PRIMARY FRONTEND-BACKEND INTEGRATION ROUTES

D. Responsive Design and Interaction Handling

The frontend incorporates responsive behavior through Tailwind CSS breakpoints: the desktop sidebar becomes a mobile slide-over drawer, dashboard KPI grids adapt to screen size, report cards use responsive grids, wide tables support horizontal scrolling, and assessment controls remain usable on smaller displays. React Hot Toast provides immediate feedback for uploads, assessment failures, and report deletion, while explicit loading, error, empty, and success states keep asynchronous operations from appearing incomplete.

VI. EVALUATION METHODOLOGY AND RELIABILITY

A. Functional Evaluation

The strongest evidence currently available is demonstration-level functional evidence rather than statistical validation. A supplied DPR demonstration was processed through the pipeline; eight sections were detected and the system produced an overall completeness score of 57.29, with section-level results including Introduction at 33.33, one Technical section at 100.00, another Technical section at 75.00, and Financial at 50.00, together with present and missing checklist items and evidence snippets. These results demonstrate that the pipeline executes its major stages and produces interpretable section-level outputs for the demonstrated document, but they do not establish generalized accuracy across a representative collection of DPRs.

B. Reliability Assessment

Reliability is best considered across repeatability, semantic correctness, and predictive validity. Deterministic components—PDF extraction, keyword evidence detection, checklist scoring, feature construction, and risk rules—are highly repeatable for identical inputs. The semantic classifier and quality assessor depend on Gemini and are promising but not formally verified against human-labeled ground truth, and predictive validity is not empirically established because the risk engine is not trained against actual project outcomes; a High-risk label therefore cannot currently be read as a validated probability of delay or failure.

Dimension	Current Assessment
Software architecture	Good
Modularity	Good
Explainability	Good
Deterministic repeatability	Good
Document completeness screening	Moderate–Good

Dimension	Current Assessment
Semantic classification	Moderate; not formally validated
LLM quality assessment	Useful but subjective
Government compliance validation	Limited
Numerical DPR validation	Weak / absent
Risk scoring	Moderate, heuristic indicator
Predictive risk modeling	Not yet implemented
Empirical reliability evidence	Insufficient
Prototype readiness	Good
Production decision-support readiness	Requires human review

TABLE III. RELIABILITY ASSESSMENT OF THE CURRENT IMPLEMENTATION

Several architectural limitations should be acknowledged. Section extraction depends heavily on the existence and correctness of a Table of Contents, so DPRs without a TOC, with incorrect page numbers, nested subsections, or unusual formatting may reduce section-detection reliability. Keyword-based completeness detection can identify that a concept is mentioned without proving it has been adequately addressed, and the system does not yet perform deterministic cross-section consistency checking or independently validate numerical calculations such as total project cost, unit rates, or timeline arithmetic. Configured section-level checklist weights are currently loaded but not applied to the overall completeness score, which is instead derived by averaging detected-section scores—making the current metric a measure of detected-section checklist completeness rather than complete-document compliance against a required-section specification.

VII. RESULTS AND DISCUSSION

A. Functional Outcomes

The implementation successfully establishes the complete core application lifecycle: official login, server-side session creation, protected dashboard access, DPR upload, file validation and storage, metadata persistence, AI assessment invocation, structured JSONB result storage, dashboard statistics, completed-report retrieval, detailed assessment viewing, and DPR/report deletion. The hybrid architecture balances semantic flexibility with deterministic control: Gemini is used where semantic interpretation is valuable, while deterministic modules handle checklist evidence, feature construction, and numerical risk calculation, improving traceability because intermediate features and risk drivers can be inspected independently.

B. Practical Impact

For reviewers, the system reduces repetitive preliminary screening by bringing section detection, completeness evidence, qualitative quality assessment, and risk indicators into one workflow. For decision-support applications, the dashboard and detailed reports provide a standardized view of assessment results, and the modular backend with a structured JSONB result model allows individual analytical components to be replaced or extended without redesigning the application. The system should nevertheless remain a decision-support aid rather than an autonomous approval mechanism, since semantic quality assessment is still subjective, formal compliance validation is incomplete, and the risk score has not been statistically validated against historical project outcomes.

VIII. LIMITATIONS AND FUTURE WORK

A. Limitations

The primary limitation is the absence of a historical outcome dataset for trained risk prediction, so the risk engine remains deterministic and transparent but not a learned probability of project failure. The system also lacks OCR, layout-aware extraction, table reconstruction, and image interpretation, which limits processing of scanned or highly visual DPRs, and it depends on TOC structure, keyword-based completeness evidence, and subjective LLM quality scoring, without numerical financial validation, deterministic cross-section consistency analysis, or a statistically meaningful benchmark dataset.

B. Future Work

The next development step is a labeled DPR evaluation dataset, with expert annotations for section detection and boundaries, semantic categories, completeness checklist items, quality scores across the six dimensions, and historical outcomes such as delay, cost overrun, and approval issues. With such a dataset, supervised models such as XGBoost, LightGBM, Random Forest, or Logistic Regression [1]–[3] could be evaluated for risk prediction using accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC, calibration, and cross-validation, while quality assessment could be compared with expert ratings using MAE, RMSE, and inter-rater agreement. The pipeline can also be extended with OCR, table extraction, layout-aware parsing, cross-section consistency analysis, authoritative MDoNER guideline retrieval, clause-level compliance checking, and a domain-specific retrieval-augmented generation layer [5].

IX. CONCLUSION

This paper presented an AI-assisted system for automated Detailed Project Report quality assessment and document-based risk analysis, integrating PDF text extraction, Table-of-Contents-based structural detection, semantic section classification, deterministic completeness assessment, LLM-based quality evaluation, structured feature engineering, explainable rule-based risk scoring, and natural-language risk explanation within a single pipeline. The implemented application extends this analytical pipeline into a complete user-facing platform with FastAPI backend services, PostgreSQL persistence, authenticated access, DPR upload and management, dashboard analytics, report retrieval, filtering, deletion, and detailed visualization through a React frontend, demonstrating that LLM capabilities can be combined with deterministic rules to improve transparency and control in document-intelligence applications.

The current implementation should be regarded as a functional prototype and preliminary decision-support system rather than a statistically validated predictive model. Its strongest contribution is the integration of document-structure understanding, evidence-based completeness assessment, qualitative quality evaluation, structured feature engineering, and explainable risk scoring; future work based on a labeled historical DPR dataset can provide the empirical foundation required for genuine risk prediction and stronger MDoNER-specific compliance assessment.

ACKNOWLEDGMENT

The authors acknowledge the project team and the academic and technical environment that supported the

development, integration, testing, and evaluation of the AI-powered DPR quality assessment and risk analysis system.

REFERENCES

- [1] A. Gondia, A. Siam, W. El-Dakhakhni, and A. H. Nassar, "Machine learning algorithms for construction projects delay risk prediction," *Journal of Construction Engineering and Management*, vol. 146, no. 1, Art. no. 04019085, Jan. 2020.
- [2] S. Alsulamy, "Predicting construction delay risks in Saudi Arabian projects: A comparative analysis of CatBoost, XGBoost, and LGBM," *Expert Systems with Applications*, Art. no. 126268, 2025.
- [3] R. Al Mnaseer, "Ensemble machine learning for risk assessment in construction project management: Applications in infrastructure development," *Asian Journal of Civil Engineering*, vol. 27, no. 2, pp. 485–497, Feb. 2026.
- [4] E. S. Ortigosa, T. Gonçalves, and L. G. Nonato, "eXplainable artificial intelligence (XAI)—from theory to methods and applications," *IEEE Access*, vol. 12, pp. 80799–80846, 2024.
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.